

PA1 Report: RTSSP and SHP

Diogo Jesus¹, Nuno Nogueira²

¹ n° 59969.

² n° 60599.

Keywords: RTSSP, SHP, secure streaming, UDP transport security, authenticated encryption, message integrity, replay protection, ECDSA signatures, ECDH key agreement, public key certificates.

1 Part 1: RTSSP Secure Streaming Protocol

1.1 Design and Packet Format

Each UDP datagram is wrapped in a 5-byte RTSSP header followed by a protected payload:

TYPE (1 B)	VERSION (2 B)	LENGTH (2 B)	payload ...
0x17	0x00 0x03	payload bytes	...

The first 8 bytes of every plaintext payload are a 64-bit sequence number. The remaining bytes are the media frame. The payload layout depends on the cipher suite:

- **AEAD** (GCM, ChaCha20-Poly1305): `encrypt(seqNum[8] | data)`, the cipher authenticates internally.
- **MAC**: `encrypt(seqNum[8] | data) | MAC(ciphertext)`, Encrypt-then-MAC; MAC is verified before decryption.
- **Hash**: `encrypt(seqNum[8] | data | hash(data))`, hash carried inside the ciphertext.

1.2 Cryptographic Configuration and Implementation

The crypto behaviour is defined by a key-value configuration file. The relevant fields are `CONFIDENTIALITY` (cipher algorithm), `SEED` (hex-encoded key material), `KEY_SIZE` (optional, in bits), `INTEGRITY` (NULL, MAC, or H), `MAC` (MAC algorithm), and `H` (hash algorithm).

Keys are never used directly from the configuration. `CipherSuiteFactory` derives the cipher key and MAC key from `SEED` via HKDF (SHA-256) with domain separation (`info="cipher-key"` and `info="mac-key"`). In Part 2, the ECDH shared secret takes the role of the seed, so the same derivation path is used in both parts. For each packet, a fresh IV or nonce is generated with `SecureRandom` and prepended to the ciphertext. GMAC tags follow the same approach, prepending a fresh 12-byte nonce to the tag.

1.3 Security Properties

Confidentiality is provided by encrypting every frame before transmission. Integrity and data-origin authentication are enforced by AEAD or an explicit MAC. Packets that fail verification are silently dropped. Replay protection uses a 64-bit sequence number embedded inside the ciphertext: the receiver maintains a set of seen sequence numbers and rejects duplicates.

1.4 Tests and Results

Unit tests cover the config parser, cipher suite construction, and RTSSP send/receive roundtrips, including replay and tampering rejection. `FixedConfigTest` runs across 35 static configurations, `SharedConfigTest` covers additional configurations where key material is derived dynamically, and `SecureSocketBaseTest` tests the packet framing layer directly.

The full pipeline was run with `movies/cars.dat`:

```
DONE! all frames sent: 30515
Movie duration 128 s
Throughput 238 fps
Throughput 2509 Kbps
```

2 Part 2: SHP Secure Handshake Protocol

2.1 Design Overview

SHP is a three-message handshake over TCP that runs before the RTSSP stream starts. It provides mutual authentication via X.509 certificates and ECDSA signatures, forward secrecy via ephemeral ECDH key pairs generated fresh per handshake, and freshness guarantees via bidirectional nonces with a challenge-response step.

2.2 Message Formats and Protocol Flow

SHP exchanges three serialised `ShpMessage` objects:

1. **CLIENT_HELLO**: requested movie name, X.509 certificate, ephemeral ECDH public key, supported cipher suite list, 16-byte nonce, and an ECDSA signature over all fields.
2. **SERVER_HELLO**: X.509 certificate, ephemeral ECDH public key, selected suite name, 16-byte server nonce, `clientNonce+1`, ECDSA signature over all fields.
3. **CSSP** (Client Session Setup Payload): `encrypt(serverNonce+1|udpPort)`, optionally with an integrity proof, using the session key derived from the ECDH shared secret.

The server may also send a **SERVER_ERROR** message to notify the client of failures it can legitimately act on, such as an unrecognised movie name or no common cipher suite. Security-sensitive failures such as an invalid signature or a repeated nonce cause the server to close the connection silently, without sending an error, to avoid leaking information to an attacker.

After the Hello messages, both sides compute `sharedSecret=ECDH(myPriv, peerPub)` using the peer's ECDH public key. This shared secret is then passed to `CipherSuiteFactory.fromConfig(suite, sharedSecret)` to obtain the `CipherSuite` used for the CSSP message and later for the RTSSP stream.

2.3 Certificates, ECDSA and ECDH

Both parties hold a PKCS12 keystore containing their private key and X.509 certificate, and a PKCS12 truststore containing the peer's certificate. These are generated by `scripts/generate-shp-stores.sh` using `keytool` with `secp256r1` (P-256) keys and `SHA256withECDSA` as the signature algorithm, provided by BouncyCastle.

During the handshake, each party signs its Hello message using its private key, and verifies the peer's signature against the certificate included in that message. Certificate trust is checked by looking up the peer certificate in the local truststore (`CertificateUtils.isTrusted`).

Key agreement uses ephemeral ECDH (`EcdhKeyAgreement`, `secp256r1`). Each `ShpServer` and `ShpClient` instance creates a fresh `ShpCryptoSpec`, which in turn creates a new `EcdhKeyAgreement` object with a newly generated ephemeral key pair. This means the ECDH private key is discarded after each handshake, providing forward secrecy: compromising a long-term signing key does not expose past session keys.

Cipher suite negotiation is server-preference ordered. The Proxy advertises the cipher suites it supports, while the Streaming Server selects the first compatible suite from its own ordered list. These server-side preference lists are configured per movie, allowing different media resources to use different cryptographic policies.

2.4 Integration with RTSSP

The `CipherSuite` produced by SHP is passed directly to `SecureDatagramSocket` or `SecureMulticastSocket`. From that point on, the framing and security properties are identical to Part 1. There is no static `crypto.properties` file for the session; suite configuration lives inline in `cipher-suites.conf` and all key material is derived from the ECDH shared secret at runtime.

2.5 Tests and Results

SHP correctness is verified by several test classes in `lib/src/test/java/shp/`. `ShpTest` runs an end-to-end handshake between an in-process client and server and then uses the resulting `CipherSuite` to perform a full RTSSP send/receive roundtrip. `ShpSuiteNegotiationTest` exercises suite selection across mismatched client and server preference lists, including cases where no common suite exists. `ShpMessagesTest` checks serialisation roundtrips for all three message types, and `ShpCryptoSpecTest` verifies ECDSA sign/verify, shared secret agreement, and nonce size.

The full Part 2 program was also executed end-to-end using `movies/cars.dat`. The Proxy completed the SHP handshake with the Streaming Server, obtained a session cipher suite, and received the entire movie stream without errors, producing the same throughput figures as Part 1.

Handshake latency was measured using `ShpHandshakeTimingTest`, which runs 20 complete handshakes per cipher suite. Representative results are shown in Table 1.

Cipher Suite	Min (ms)	Avg (ms)	Max (ms)
AES/GCM/NoPadding	55	72	121
ChaCha20-Poly1305	55	70	118
AES/CBC/PKCS5Padding + HMAC-SHA256	56	71	120

Table 1: SHP handshake latency over 20 iterations per suite (loopback, `secp256r1`).

The results confirm that the dominant cost is the ECDH and ECDSA computation, not the symmetric cipher. All three suites produce essentially the same latency, which makes sense because the bulk encryption performed during SHP is limited to the small CSSP payload.

3 Limitations and Assumptions

The trust model follows the assignment: endpoints, JVM, OS, and BouncyCastle are trusted. Threats are limited to the network channel. The Part 1 session setup uses a plain TCP exchange to request a movie and confirm the multicast address. This unprotected exchange is replaced by SHP in Part 2. SHP does not support session resumption and handles one client at a time.

The truststore-based model is sufficient since both parties are pre-configured with each other's certificate.

4 Conclusion

This project implements two complementary security layers for a UDP-based real-time media streaming system. Part 1 introduces RTSSP, which provides confidentiality, integrity, data-origin authentication, and replay protection for every streamed frame using a static cryptographic configuration. Part 2 extends this with SHP, a lightweight handshake protocol that adds mutual authentication, ephemeral key agreement, and dynamic cipher suite negotiation before the stream begins.

The implementation was validated with unit tests covering the cryptographic components, packet protection logic, SHP message handling, cipher suite negotiation, and end-to-end integration. Manual streaming executions were also used to confirm that the negotiated security configuration can be used to deliver the protected movie stream successfully.